

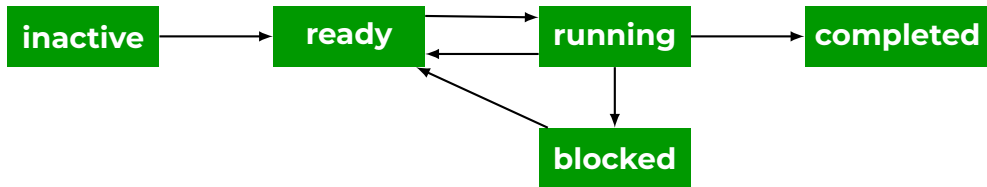
Programación concurrente y paralela

Práctica 2 - Semáforos

Primer Cuatrimestre 2026



Repaso



Hasta ahora estábamos resolviendo el problema de la exclusión mutua haciendo *busy waiting*. Los semáforos nos permiten usar el estado *blocked* del sistema operativo.

Son un tipo abstracto con una cantidad de permisos, y métodos para dar un permiso y para obtener un permiso. Los nombres pueden variar aún cuando el funcionamiento es el mismo:

- `acquire()` y `release()` (Herlihy, Java)
- `wait()` y `signal()` (Ben-Ari)
- `P()` y `V()` (Dijkstra, del holandés *Prolaag* (probar) y *Verhoog* (subir))

La semántica de estos métodos es:

- **Acquire**: Tomar un permiso, si no hay permisos disponibles entrar en el conjunto de threads que están esperando.
- **Release**: Devolver un permiso, incrementando la cantidad si nadie está esperando y despertando a un thread de los que esperan si los hay

La forma pura de usar los semáforos es por medio de estos dos métodos. Hay implementaciones con un método para preguntar la cantidad de permisos de un semáforo (ej. `availablePermits` en Java), pero **no deben usarse para sincronización**

En términos del orden para despertar threads, existen dos variantes:

- **Semáforos fuertes:** Los threads esperando en el semáforo están en una cola y se despiertan en el orden de la misma (FIFO)
- **Semáforos débiles:** Los threads esperando en el semáforo están en un conjunto y cualquiera podría ser despertado.

La implementación fuerte tiene la ventaja de que elimina la posibilidad de *starvation* (esto puede ser salvable, lo veremos pronto).

Sin embargo, los semáforos por default son débiles. Dos motivos para esto son:

- A veces es más eficiente despertar a un thread que haya estado ejecutando recientemente porque se aprovecha mejor la caché.
- En muchas implementaciones la selección del thread a despertar se hace con ayuda del sistema operativo, usando la información de scheduling (ej. prioridad de los threads)

Si un thread debe realizar una acción antes que otro, podemos denotar la realización de esa acción con un semáforo inicializado en 0

```
global Semaphore accionRealizada = Semaphore(0)
```

```
thread p
    //antes de la accion
    accionRealizada.release()
```

```
thread q
    accionRealizada.acquire()
    //despues de la accion
```


Si tenemos un recurso compartido que queremos usar de manera limitada, podemos establecer un semáforo inicializado con tantos permisos como usos concurrentes admite el recurso.

```
global Semaphore aLoSumoDos = Semaphore(2)
```

```
thread p
```

```
    aLoSumoDos.acquire()  
    //usar el recurso  
    aLoSumoDos.release()
```

```
thread q
```

```
    aLoSumoDos.acquire()  
    //usar el recurso  
    aLoSumoDos.release()
```

```
thread r
```

```
    aLoSumoDos.acquire()  
    //usar el recurso  
    aLoSumoDos.release()
```

Supongamos un recurso compartido tal que cada thread necesita más de uno de ese elemento. Potencialmente hay una cantidad suficiente como para que un thread tome todas las copias necesarias de ese recurso, pero como sólo consideramos atómico tomar *una* copia, corremos riesgo de que los threads formen un deadlock.

```
global Semaphore aLoSumoDos = Semaphore(2)
```

```
thread p
    aLoSumoDos.acquire()
    aLoSumoDos.acquire()
    //usar los recursos
    aLoSumoDos.release()
    aLoSumoDos.release()
```

```
thread q
    aLoSumoDos.acquire()
    aLoSumoDos.acquire()
    //usar los recursos
    aLoSumoDos.release()
    aLoSumoDos.release()
```

Dos threads donde cada uno habilita la ejecución del otro. El efecto obtenido es que la ejecución queda intercalada. El valor inicial de los semáforos determina quién empieza.

```
global Semaphore puedeq = Semaphore(1)
global Semaphore puedep = Semaphore(0)
```

```
thread p
    puedep.acquire()
    //accion p
    puedeq.release()
```

```
thread q
    puedeq.acquire()
    //accion q
    puedep.release()
```

Problemas clásicos con Semáforos

Hay thread *productores*, que escriben en un buffer (un array), y threads *consumidores*, que leen del buffer.

```
global Semaphore notEmpty = Semaphore(0)
global Semaphore notFull = Semaphore(N)
global Semaphore mutexP = Semaphore(1)
global Semaphore mutexC = Semaphore(1)
```

```
thread consumidor
    Dato d
    while(true) {
        notEmpty.acquire()
        mutexC.acquire()
        d = buffer[fin]
        fin = (fin + 1) % N
        mutexC.release()
        notFull.release()
        consume(d)
    }
```

```
thread productor
    Dato d
    while(true) {
        d = produce()
        notFull.acquire()
        mutexP.acquire()
        buffer[inicio] = d
        inicio = (inicio + 1) % N
        mutexP.release()
        notEmpty.release()
    }
```

Hay una clase de threads *lectores* que pueden usar el recurso concurrentemente de manera no acotada, y *escritores*, que deben tener exclusividad sobre el recurso. Se puede solucionar con prioridad para los lectores y para los escritores.

Prioridad lectores:

```
global int cantLectores = 0
global Semaphore mutexL = Semaphore(1)
global Semaphore escribir = Semaphore(1)
```

```
thread lector
while(true) {
    mutexL.acquire()
    cantLectores++
    if (cantLectores==1)
        escribir.acquire()
    mutexL.release()
    // leer
    mutexL.acquire()
    cantLectores--
    if (cantLectores==0)
        escribir.release()
    mutexL.release()
}
```

```
thread escritor
while(true) {
    escribir.acquire()
    // escribir
    escribir.release()
}
```

La solución anterior tiene riesgo de starvation para los escritores, si los lectores siempre llegan a la sección de lectura antes de que esta se vacíe. Esto se puede mejorar utilizando un semáforo adicional `molinete` que los lectores obtienen e inmediatamente después liberan. Haciendo que los escritores también tomen `molinete` y haciéndolo fuerte, se puede garantizar un lugar para la ejecución de los escritores.


```
global Semaphore molinete = Semaphore(1,true)
```

```
thread lector
while(true) {
    molinete.acquire()
    molinete.release()
    mutexL.acquire()
    cantLectores++
    if (cantLectores==1)
        escribir.acquire()
    mutexL.release()
    // leer
    mutexL.acquire()
    cantLectores--
    if (cantLectores==0)
        escribir.release()
    mutexL.release()
}
```

```
thread escritor
while(true) {
    molinete.acquire()
    escribir.acquire()
    // escribir
    molinete.release()
    escribir.release()
}
```

La otra posibilidad es darle la prioridad a los escritores: Si hay escritores esperando acceder al recurso, directamente se impide el acceso a los lectores.

Variables globales:

```
global int cantLectores = 0
global int escritoresEsperando = 0
global Semaphore mutexL = Semaphore(1)
global Semaphore mutexE = Semaphore(1)
global Semaphore mutexP = Semaphore(1)
global Semaphore leer = Semaphore(1)
global Semaphore escribir = Semaphore(1)
```

```
thread lector
while(true) {
    mutexP.acquire()
    leer.acquire()
    mutexL.acquire()
    cantLectores++
    if (cantLectores==1)
        escribir.acquire()
    mutexL.release()
    leer.release()
    mutexP.release()
    // leer
    mutexL.acquire()
    cantLectores--
    if (cantLectores==0)
        escribir.release()
    mutexL.release()
}
```

```
thread escritor
while(true) {
    mutexE.acquire()
    escritoresEsperando++
    if (escritoresEsperando == 1)
        leer.acquire()
    mutexE.release()

    escribir.acquire()
    // escribir
    escribir.release()

    mutexE.acquire()
    escritoresEsperando--
    if (escritoresEsperando == 0)
        leer.release()
    mutexE.release()
}
```

Si queremos resolver el problema de exclusión mutua con semáforos, podemos usar un semáforo binario (con a lo sumo 1 permiso).

```
global Semaphore mutex = Semaphore(1)
```

```
thread p
while(true) {
    //SNC
    mutex.acquire()
    //SC
    mutex.release()
    //SNC
}
```

```
thread q
while(true) {
    //SNC
    mutex.acquire()
    //SC
    mutex.release()
    //SNC
}
```

¿Qué pasa con N threads? Si mutex es débil, hay chance de *starvation*.

¿Será posible resolver el problema si no disponemos de semáforos fuertes?

Dijkstra se encontró con el problema y conjeturó que **no es posible**.

Sin embargo, en 1979 Morris propuso un algoritmo para resolver el problema. En 1986 Udding presentó un algoritmo con el mismo concepto pero algo simplificado.

La idea es tener dos “compuertas” Los threads que quieren ejecutar su sección crítica deben pasar por la primera compuerta y luego por la segunda, pero hay un solo permiso compartido entre las compuertas, y no se asigna a la segunda. Se usa un semáforo adicional *mutex* para evitar que un thread pueda quedar en *starvation* intentando pasar la puerta 1.

```
1 global int threadsP1 = 0
2 global int threadsP2 = 0
3 global Semaphore puerta1 = Semaphore(1)
4 global Semaphore puerta2 = Semaphore(0)
5 global Semaphore mutex = Semaphore(1)
6
7 thread p
8     while(true) {
9         puerta1.acquire()
10        threadsP1++
11        puerta1.release()
```

```
12        mutex.acquire()
13        puerta1.acquire()
14        threadsP1--
15        threadsP2++
16        if (threadsP1>0)
17            puerta1.release()
18        else
19            puerta2.release()
20        mutex.release()
21        puerta2.acquire()
22        threadsP2--
23        //SC
24        if (threadsP2>0)
25            puerta2.release()
26        else
27            puerta1.release()
28    }
```


Notar que hay un solo permiso que puede estar en `puerta1`, `puerta2` o en posesión de un thread, no es posible que dos threads estén ejecutando su sección mutua concurrentemente.

Asimismo, todo bloque de código en posesión de un semáforo es código de entrada, salida, o la sección crítica, por lo que asumiendo que esta termina, el semáforo siempre vuelve a alguno de los `puertas`, procurando el progreso.

La clave que permite afirmar que se evita la starvation es que los threads tienen que vaciar la primera etapa para iniciar la segunda (notar que el semáforo `mutex` es importante para no dejar a un thread impedido de incrementar `threadsP1`)

Otros problemas clásicos de Semáforos

Presentado por Suhas Patil en 1971.

- Comenzamos con seis threads: tres threads *Agentes* y tres threads *Fumadores*.
- Los fumadores están en un loop infinito armando cigarrillos, que requieren 3 elementos: tabaco, papel de armar, y fósforos.
- Cada uno de los 3 fumadores posee una cantidad infinita de uno de los elementos (uno tiene infinito tabaco, uno infinito papel, y uno infinitos fósforos).
- Los agentes, también en un loop infinito, proporcionan dos de los tres elementos (Uno da tabaco y papel, uno papel y fósforos y uno tabaco y fósforos), una unidad de cada uno por cada iteración.

- Hay un semáforo que representa la disponibilidad de cada uno de los elementos (comienzan en 0)
- Hay un semáforo `activarAgente`, que comienza con 1 permiso. Cada agente libera sus recursos sólo una vez consumido un permiso de `activarAgente`.

Código de uno de los agentes:

```
1  thread AgenteTP { //proporciona tabaco y papel
2      while (true) {
3          activarAgente.acquire()
4          tabaco.release()
5          papel.release()
6      }
7  }
```

El objetivo es programar a los fumadores (o threads adicionales de ser necesario) **sin modificar a los agentes**, de manera que que el fumador adecuado pueda aprovechar los ingredientes liberados, armar el cigarrillo, y habilitar una nueva entrega.

La propuesta “ingenua” para resolver este problema sería tener, para cada fumador, el thread:

```
1  thread fumadorT { //Tiene tabaco
2      while(true) {
3          papel.acquire()
4          //Tomar papel
5          fosforos.acquire()
6          //Tomar fosforos
7          //Armar cigarrillo
8          activarAgente.release()
9      }
10 }
```

¿Cuál es el problema de este esquema? Puede derivar en un Deadlock

Vamos a escribir tres threads *gestores* (uno por cada recurso) y vamos a hacer que cada uno

1. Si encuentra que además del recurso que acaba de tomar, ya hay otro disponible, despierta al thread que necesita los dos recursos disponibles (usamos semáforos `fumadorFosforo`, `fumadorTabaco` y `fumadorPapel` inicializados en 0)
2. Si no, meramente marca que se puso disponible el recurso

Esto deberá suceder en exclusión mutua con los otros gestores (usamos un semáforo `mutexGestores`)

Declaramos variables `hayTabaco`, `hayPapel` y `hayFosforo` inicializadas en `false` y hacemos:

```
1  thread gestorT { //Verifica el tabaco
2      while(true) {
3          tabaco.acquire()
4          mutexGestores.acquire()
5          if hayPapel {
6              fumadorFosforo.release()
7          } elseif hayFosforo {
8              fumadorPapel.release()
9          } else {
10             hayTabaco = true
11         }
12         mutexGestores.release()
13     }
14 }
```


Luego, el código de cada fumador es sencillo:

```
1  thread fumadorT { //Tiene tabaco
2      while(true) {
3          fumadorTabaco.acquire()
4          //Tomar papel y fosforo
5          hayPapel = false
6          hayFosforo = false
7          //Armar cigarrillo
8          activarAgente.release()
9      }
10 }
```

Propuesto por Dijkstra en 1965.

Una peluquería tiene n asientos para esperar y un barbero, que cuando no le piden un corte se queda dormido. Si cuando un cliente llega no hay nadie esperando, el cliente despierta al barbero y se atiende inmediatamente; si no, intenta tomar un asiento. Si a la llegada del cliente los n asientos están ocupados, este **se va sin atenderse**.

En este caso los clientes tienen un método `cortarseElPelo()` y el barbero con un método `cortarPelo()`, estos métodos se deben ejecutar concurrentemente.

Para contar la gente en la barbería, en este caso vamos a usar un contador entero con un mutex. Más aún, el contará será por los clientes en la barbería (y sumará hasta $n + 1$). Esto es para no tener que separar el caso donde alguien se está atendiendo pero todos los asientos están vacíos.

Para coordinar el corte, usamos cuatro semáforos: El primer par es para coordinar el inicio del corte, el segundo para coordinar la finalización.

El semáforo con el que el barbero habilita el siguiente corte debe ser fuerte si se quiere preservar el orden para atender a los clientes.

Variables a usar:

```
1 global int genteEnBarberia = 0
2
3 global Semaphore solicitudCorte = Semaphore(0)
4 global Semaphore listoParaCortar = Semaphore(0, true)
5 global Semaphore corteTerminado = Semaphore(0)
6 global Semaphore clienteRetirado = Semaphore(0)
7 global Semaphore mutexClientes = Semaphore(1)
```

```
thread Cliente
    bool meVoy = false
    mutexClientes.acquire()
    if (genteEnBarberia == n+1)
        meVoy = true
    else
        genteEnBarberia++
    mutexClientes.release()
    if (!meVoy){
        solicitudCorte.release()
        listoParaCortar.acquire()
        cortarseElPelo()
        corteTerminado.acquire()
        mutexClientes.acquire()
        genteEnBarberia--
        mutexClientes.release()
        clienteRetirado.release()
    }
```

```
thread Barbero
    while(true) {
        solicitudCorte.acquire()
        listoParaCortar.release()
        cortarPelo()
        corteTerminado.release()
        clienteRetirado.acquire()
    }
```

¿Hay manera de preservar el orden *sin* usar semáforos fuertes?

De hecho, sí. El truco es hacer un esquema productores-consumidores donde el dato *son semáforos*. Cada posición del buffer corresponde con un lugar en la barbería. El `notEmpty` de P/C se corresponde con el `solicitudCorte`, y en vez de usar un `notFull`, tenemos que tener el contador de posiciones ocupadas en el buffer.

Cada cliente llega, declara un semáforo local, lo pone (como referencia) en el buffer, y luego se pone a esperarlo. El barbero va habilitando clientes tomando semáforos del buffer y haciéndoles `release()`. Como el consumo del buffer es FIFO, se preservó el orden de los clientes.

-  Mordechai Ben-Ari
Principles of Concurrent and Distributed Programming, Capítulo 6
Prentice Hall, 2006.
-  Maurice Herilhy, Nir Shavit
The Art of Multiprocessor Programming (2nd Ed), Sección 8.5
Morgan Kaufmann, 2020.

Laboratorio

Creación de Threads

Main.java

```
public class Main {  
    public static void  
        main(String[] args) {  
        // ...  
    }  
}
```



- Cuando se corre este fragmento de código, ya se está ejecutando dentro de un thread: el *main thread*.
- Cualquier programa que no crea threads a propósito corre entero ahí, en un solo thread.
- ¿Cómo hacemos para crear nuevos threads que corran concurrentemente?

Saludo.java

```
class Saludo extends Thread {  
    public void run() {  
        System.out.println(  
            "Hola desde "  
            + getName());  
    }  
}  
  
new Saludo().start();
```



- Para Java, cualquier thread de ejecución (ya sea un thread tradicional del sistema operativo o un thread virtual moderno) está representado por un objeto de tipo `java.lang.Thread`.
- Este objeto es el punto de control que utiliza la JVM para administrar su ciclo de vida.
- Por sí solo, un **Thread** no hace nada: para decirle qué ejecutar hay que crear una clase nueva que sobrescriba (*override*) el método `run()`.

Antes de start()

Un objeto Thread es una instancia como cualquier otra: no tiene nada de especial. Recién cuando se invoca `start()` empieza a existir un thread de ejecución real.

- Ese thread del SO arranca corriendo el `run()` de nuestro objeto.
- Es *asincrónico*: `start()` retorna enseguida, no espera a que el thread termine.
- Se asocia con un Thread del kernel. Es el modelo **1:1** que ya se vio la clase pasada.

Runnable.java

```
public interface Runnable {  
    void run();  
}
```



- Es una interfaz de `java.lang` con un único método abstracto: `void run()`.
- Representa solamente *la tarea*: no sabe nada de threads ni de cómo se va a ejecutar.
- Al tener un solo método, es una *interfaz funcional*: se puede implementar con una lambda o un method reference, sin declarar una clase.

Saludo.java

```
class Saludo implements Runnable {  
    public void run() {  
        System.out.println(  
            "Hola desde "  
            + Thread.currentThread()  
                .getName());  
    }  
}  
  
new Thread(new Saludo()).start();
```



- Thread tiene un constructor que recibe algo que implemente la interfaz **Runnable**:
Thread(Runnable target).
- Por dentro, **Thread** guarda ese **Runnable** como **target**, y su propio **run()** simplemente llama a **target.run()**.

Method reference

```
1 static void tarea() {  
2     // ...  
3 }  
4  
5 new Thread(Demo::tarea).start();
```

Lambda

```
1 new Thread(() -> tarea()).start();  
2  
3 // o inline, si es trivial:  
4 new Thread(() -> System.out  
5     .println("hola")).start();
```

- **Method reference:** apunta a un método que ya existe, cómodo si la lógica no es trivial.
- **Lambda:** igual, pero inline, cómodo para lógica corta que no amerita un método aparte.

La lambda no tiene un tipo por sí misma, el compilador se lo infiere del contexto (*target typing*).

`Thread(Runnable target)` espera un `Runnable`. Como `Runnable` es una interfaz funcional y la lambda calza con su única firma (`run()`, sin parámetros, sin retorno), el compilador genera automáticamente un objeto que la implementa:

```
1 // () -> tarea() es equivalente a:  
2 new Runnable() {  
3     public void run() { tarea(); }  
4 }
```

La lambda no es un `Runnable`, se *convierte* en uno porque el contexto (acá, el parámetro del constructor) esperaba uno. Un method reference como `Demo::tarea` funciona con la misma lógica.

- **extends Thread**: la clase es un Thread. Java tiene herencia simple, si ya extendés Thread no podés extender ninguna otra clase.
- **Runnable**: no consume esa herencia. La clase que implementa la tarea puede heredar de cualquier otra cosa que necesite, y el mismo objeto se puede pasar a más de un Thread.

En la práctica

Por eso Runnable (clase separada, method reference o lambda) es casi siempre la opción preferida. `extends Thread` queda para los casos donde de verdad hace falta sobrescribir otro comportamiento del propio Thread, no solo `run()`.

.run() en vez de .start()

Compila y no tira error, pero ejecuta *sincrónicamente* en el thread que llama, sin crear ningún thread nuevo.

Un Thread es de un solo uso

Llamar `start()` dos veces sobre el mismo objeto tira `IllegalThreadStateException`. Un `Runnable`, al ser solo lógica, se puede envolver en un `Thread` nuevo cada vez que haga falta correrlo de nuevo.

Esperando threads: join

join()

Es una herramienta de sincronización que le permite a un thread (por lo general, el thread principal o `main`) pausar su propia ejecución hasta que el thread al que se lo aplicas termine por completo.



Main.java

```
static int count = 0;  
  
Thread t = new Thread(  
    () -> count++);  
t.start();  
t.join();  
System.out.println(count);
```

try-catch

```
try {  
    t.join();  
} catch (InterruptedException e) {  
    // ...  
}
```

throws

```
public static void main(  
    String[] args)  
    throws Exception {  
    t.join();  
    // ...  
}
```

join() puede tirar InterruptedException, que es *checked*: hay que atenderla sí o sí, con try-catch o declarándola con throws. Lo vemos en detalle más adelante.

Mal: serializa todo

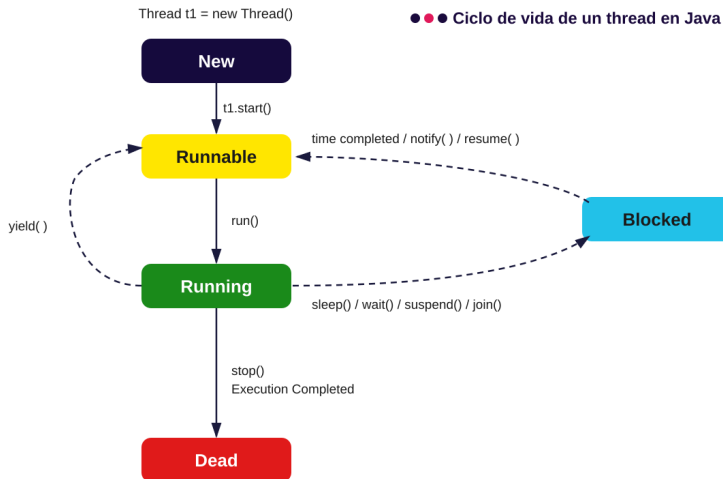
```
1 for (int i = 0; i < N; i++) {  
2     Thread t = new Thread(...);  
3     t.start();  
4     t.join(); // espera ACA  
5 }
```

Ya no hay concurrencia real: es un programa secuencial con threads de más.

Bien: dos loops

```
1 Thread[] ts = new Thread[N];  
2 for (int i = 0; i < N; i++) {  
3     ts[i] = new Thread(...);  
4     ts[i].start();  
5 }  
6 for (int i = 0; i < N; i++) {  
7     ts[i].join();  
8 }
```

Ciclo de vida de un Thread



- **New:** el objeto existe, pero todavía no se llamó `start()`.
- **Runnable:** ya puede ejecutar, esperando que el scheduler del SO le dé CPU.
- **Running:** el scheduler lo eligió y está corriendo de verdad. Puede volver a Runnable en cualquier momento (por ejemplo con `yield()`, o porque el SO le saca la CPU).
- **Blocked:** no puede seguir todavía; se entra ahí con `sleep()`, `wait()` o `join()` sobre otro thread. Se vuelve a Runnable cuando se cumple la condición (pasó el tiempo, alguien hizo `notify()`, o el otro thread terminó).
- **Dead:** `run()` terminó (o el thread murió por una excepción no atrapada). No hay vuelta atrás: no se puede volver a `start()` el mismo objeto.

Interrumpiendo Threads

thread.interrupt()

Le pide a thread que pare, prendiendo un flag booleano interno de interrupción. No lo frena a la fuerza: es cooperativo, el propio thread decide qué hacer con ese pedido.

Si el thread está corriendo código normal, el flag simplemente queda prendido hasta que el código lo consulte. Si está **bloqueado** (en `sleep()`, `wait()`, `join()`, etc.), la JVM lo despierta enseguida tirándole una `InterruptedException`.

Ya la vimos con `join()`: hay que atenderla con `try/catch` o declararla con `throws`.

Es *checked* a propósito: el compilador obliga a decidir qué hacer cuando el thread fue interrumpido mientras estaba bloqueado. Ignorarla en silencio (catch vacío) es el error más común.

Uso

```
thread.interrupt();           // pide interrumpir
thread.isInterrupted();       // consulta el flag (no lo modifica)
Thread.interrupted();         // consulta el flag del thread actual
                              // (y ademas lo apaga)
```

Cuando se tira **`InterruptedException`**, la JVM apaga el flag automáticamente: el catch arranca con el flag en **`false`**, aunque el pedido de interrupción ya haya pasado.

Mal

```
try {  
    Thread.sleep(1000);  
} catch (  
    InterruptedException e) {  
    // nada  
}
```

Bien

```
try {  
    Thread.sleep(1000);  
} catch (  
    InterruptedException e) {  
    Thread.currentThread()  
        .interrupt();  
}
```

Punto clave

Si atrapás `InterruptedException` y no hacés nada, quien te llamó no tiene forma de enterarse de que hubo un pedido de interrupción. O la propagás (`throws`), o volvé a prender el flag con `Thread.currentThread`

Volatile

Thread 1

```
x = 1;  
r1 = y;
```

Thread 2

```
y = 1;  
r2 = x;
```

Podíamos tener $r1 = 0$ y $r2 = 0$ por los reordenamientos.

Main.java

```
static volatile  
    boolean stop = false;
```



- Garantiza **visibilidad**: cuando un thread escribe una variable **volatile**, los demás threads ven ese cambio de inmediato, sin quedarse con una copia cacheada vieja.
- Se declara agregando la palabra **volatile** al tipo del campo.

Thread 1

```
static volatile int x = 0;  
x = 1;  
r1 = y;
```

Thread 2

```
static volatile int y = 0;  
y = 1;  
r2 = x;
```

El write **volatile** happens-before la lectura que lo ve: queda visible en memoria antes de esa lectura. No hay reordenamiento, y no podíamos tener $r1 = 0$ y $r2 = 0$.

Main.java

```
static volatile  
    int count = 0;  
  
// count++ en N threads  
// sigue dando mal
```



Punto clave

volatile no arregla count++: leer, sumar y escribir siguen siendo tres pasos separados, no atómicos. Visibilidad y atomicidad son dos garantías distintas.

Mutex: ReentrantLock

```
java.util.concurrent.locks.Lock
```

Una interfaz que representa un mutex: solo un thread a la vez puede tenerlo tomado. ReentrantLock es su implementación de uso general.

Declaración

```
static final Lock lock = new ReentrantLock();
```

SharedCounterLock.java

```
lock.lock();  
try {  
    count++;  
} finally {  
    lock.unlock();  
}
```

- `lock()` bloquea hasta que el thread consigue tomar el lock: simétrico a `acquire()` de un semáforo.
- `unlock()` lo libera para que otro thread pueda tomarlo.
- A diferencia de un semáforo, el lock tiene **dueño**: solo quien lo tomó lo puede liberar.

¿Por qué try/finally es obligatorio?

Mal

```
lock.lock();  
count++;  
riesgoso();  
lock.unlock();
```

Bien

```
lock.lock();  
try {  
    count++;  
    riesgoso();  
} finally {  
    lock.unlock();  
}
```

Punto clave

Si `riesgoso()` tira una excepción antes del `unlock()` de la izquierda, el lock queda tomado para siempre: los demás threads se bloquean y nunca se liberan.

Reentrancy

El mismo thread que ya tiene el lock tomado puede volver a llamar `lock()` sin bloquearse a sí mismo. Internamente se lleva la cuenta (*hold count*), y hacen falta tantos `unlock()` como `lock()` para soltarlo del todo.

Es útil cuando un método que toma el lock llama a otro método que también lo toma (por ejemplo, llamadas recursivas). Un `Semaphore(1)` **no** tiene esta propiedad: si el mismo thread llama `acquire()` dos veces, se bloquea a sí mismo.

Uso

```
if (lock.tryLock()) {  
    try {  
        // seccion critica  
    } finally {  
        lock.unlock();  
    }  
} else {  
    // no se pudo tomar el lock  
}
```

Devuelve **true/false** de inmediato: simétrico a **tryAcquire()** de un semáforo. También existe **tryLock(timeout, unit)**, con límite de tiempo.

- `lock()` ignora interrupciones mientras espera. `lockInterruptibly()` tira `InterruptedException` si el thread es interrumpido mientras espera el lock.
- `new ReentrantLock(true)` crea un lock **fair**: los threads lo obtienen en el orden en que lo pidieron (FIFO), evitando *starvation*.
- Por defecto (**fair = false**) el lock no es justo: prioriza throughput por sobre el orden de llegada. El mismo trade-off que van a ver con semáforos débiles/fuertes.

Semáforos en Java

`java.util.concurrent.Semaphore`

La implementación real del semáforo que ya conocen de la teórica. Guarda un contador de *permits* disponibles.

Constructores

```
new Semaphore(int permits);  
new Semaphore(int permits, boolean fair);
```

Uso

```
sem.acquire();  
try {  
    // seccion critica  
} finally {  
    sem.release();  
}
```

- `acquire()` bloquea hasta conseguir un permit, y tira `InterruptedException` (checked).
- `release()` devuelve un permit.
- A diferencia de `ReentrantLock`, **no hay dueño**: cualquier thread puede llamar `release()`, no hace falta que sea el que hizo `acquire()`.

Uso

```
sem.acquire(3);    // pide 3 permits de una
sem.release(3);    // devuelve 3 permits de una
```

Útil cuando una tarea necesita reservar de una varios recursos idénticos del mismo pool, en vez de pedirlos uno por uno.

Uso

```
if (sem.tryAcquire()) {  
    try {  
        // seccion critica  
    } finally {  
        sem.release();  
    }  
} else {  
    // no habia permits disponibles  
}
```

Devuelve **true/false** de inmediato. También existe **tryAcquire(timeout, unit)**, que espera hasta un límite de tiempo, y **acquireUninterruptibly()**, que bloquea como **acquire()** pero ignora interrupciones.

Mal: check-then-act

```
if (sem.availablePermits() > 0) {  
    sem.acquire(); // puede bloquear igual!  
}
```

Punto clave

Entre el `if` y el `acquire()` otro thread puede tomar el último permit: es un check-then-act roto, la combinación nunca es atómica. Para «intentar sin bloquear» ya existe `tryAcquire()`, que hace las dos cosas en una sola operación atómica.

- `new Semaphore(n) ⇒ fair = false` por defecto: semáforo **débil**, no garantiza orden de atención. Prioriza throughput.
- `new Semaphore(n, true) ⇒` semáforo **fuerte**: los threads consiguen permits en el orden en que los pidieron (FIFO), evitando *starvation*.
- Nota fina: incluso con `fair = true`, un `tryAcquire()` **sin** timeout puede colarse igual (*barging*), está documentado así a propósito.

- Un `Semaphore(1)` se parece a un mutex, pero no es lo mismo.
- **Sin dueño:** cualquier thread puede llamar `release()`, no hace falta que sea el que hizo `acquire()`.
- **No es reentrante:** si el mismo thread llama `acquire()` dos veces, se bloquea a sí mismo. Un `ReentrantLock` no tiene ese problema.

Semáforos por dentro

La pregunta

`acquire()` bloquea el thread cuando no hay permits. ¿Cómo hace Java para *bloquear un thread de verdad*, sin gastar CPU dando vueltas (*busy-waiting*)?

La respuesta tiene tres capas: **Semaphore** usa un framework interno llamado **AbstractQueuedSynchronizer** (AQS), que a su vez bloquea threads con **LockSupport.park()**, que a su vez llama al sistema operativo.

```
java.util.concurrent.locks.AbstractQueuedSynchronizer
```

Una clase base de `java.util.concurrent` para construir mutex, semáforos, latches y barreras. `Semaphore`, `ReentrantLock` y `CountDownLatch` están todos implementados encima de ella.

Provee dos cosas genéricas: un entero atómico (**state**) para representar «cuánto hay disponible», y una cola de threads esperando cuando no alcanza.

AQS (simplificado)

```
private volatile int state; // = permits disponibles

protected boolean compareAndSetState(int expect, int update) {
    return U.compareAndSetInt(this, STATE_OFFSET, expect, update);
}
```

Semaphore usa **state** para guardar los permits. Tomar un permit es un CAS: leer **state**, si es > 0 intentar bajarlo en uno con **compareAndSet**. Si otro thread se adelantó, se reintenta.

Sync interno de Semaphore (simplificado)

```
protected int tryAcquireShared(int acquires) {  
    for (;;) {  
        int available = getState();  
        int remaining = available - acquires;  
        if (remaining < 0 ||  
            compareAndSetState(available, remaining))  
            return remaining;  
    }  
}
```

Si `remaining < 0` (no había permits) el CAS ni se intenta: el thread tiene que esperar.
`tryReleaseShared` hace lo simétrico, sumando en vez de restar.

Si `tryAcquireShared` devuelve negativo, AQS encola el thread actual en una cola de espera (una lista enlazada de nodos, variante del algoritmo **CLH**) y lo bloquea.

- Cada nodo de la cola representa un thread esperando, en el orden en que llegó.
- Cuando se libera un permit, AQS recorre la cola y despierta al primero que corresponda.
- El parámetro **fair** decide si un thread que recién llega puede «colarse» antes de que le toque a la cola, o si tiene que respetar el orden.

java.util.concurrent.locks.LockSupport

```
LockSupport.park();           // bloquea el thread actual  
LockSupport.unpark(thread);   // lo despierta
```

AQS no implementa el bloqueo él mismo: cuando un thread tiene que esperar, llama a `LockSupport.park()`. Cuando otro thread libera un permit, llama a `LockSupport.unpark()` sobre el thread que estaba esperando. Este es el punto donde la JVM le pide ayuda al sistema operativo.

futex = ***fast userspace mutex***

Una primitiva de Linux pensada para que el caso común, sin contención, no necesite ni un solo llamado al kernel.

- Tomar y liberar el lock se resuelve en **espacio de usuario**, con una operación atómica (CAS) sobre una palabra en memoria compartida. Si sale bien, no hubo ninguna syscall.
- Solo en el peor caso, cuando el lock ya está tomado y hay que dormir el thread (o hay que despertar a alguien), se hace la syscall `futex(2): FUTEX_WAIT` o `FUTEX_WAKE`.
- Por eso es *rápida*: el costo de ir al kernel solo se paga cuando realmente hace falta bloquear, no en cada `lock()/unlock()`.

Cada thread de la JVM tiene asociado un objeto **Parker** nativo con un `pthread_mutex_t` y una variable de condición `pthread_cond_t` (la API estándar de POSIX threads).

- `park()`: toma el `mutex` y, si no hay nada pendiente, hace `pthread_cond_wait()`: el kernel duerme el thread sin gastar CPU hasta que alguien lo despierte.
- `unpark()`: toma el `mutex` y hace `pthread_cond_signal()` para despertar al thread dormido.
- La JVM no llama a **futex** directamente: llama a la API POSIX. Es **glibc** (NPTL) la que, por debajo de `pthread_mutex/pthread_cond`, usa la syscall **futex** para bloquear y despertar threads sin busy-waiting.

La idea es la misma, cambia la primitiva del sistema operativo.

- Windows no tiene **futex**, pero ofrece primitivas equivalentes a nivel kernel para bloquear y despertar threads sin busy-waiting: *keyed events* (**NtWaitForKeyedEvent/NtReleaseKeyedEvent**) en versiones más viejas de la JVM, o **WaitOnAddress/WakeByAddressSingle** en versiones más nuevas.
- La primitiva exacta cambia con la versión de la JVM y del sistema operativo, pero el rol es siempre el mismo: dormir un thread barato y despertarlo cuando corresponde.

1. `sem.acquire()` llama a `Sync.acquireSharedInterruptibly(1)`.
2. AQS intenta `tryAcquireShared` con un CAS sobre `state`. Si alcanza, listo, no hubo bloqueo real.
3. Si no alcanza, AQS encola el thread en su cola de espera y llama a `LockSupport.park()`.
4. `park()` baja al sistema operativo: `pthread_cond_wait` en Linux (con `futex` debajo, en `glibc`), su equivalente en Windows.
5. Cuando otro thread hace `release()`, AQS suma a `state` y llama a `LockSupport.unpark()` sobre el primero de la cola, que despierta al thread con `pthread_cond_signal` (o equivalente).